

From Reviewers' Lens: Understanding Bug Bounty Report Invalid Reasons with LLMs

Jiangrui Zheng, Yingming Zhou, Ali Abdullah Ahmad, Hanqing Yao, and Xueqing Liu[†]

Department of Computer Science

Stevens Institute of Technology

Hoboken, NJ, USA

{jzheng36, yzhou136, aahmad6, hyao9, xliu127}@stevens.edu

Abstract—Bug bounty platforms (e.g., HackerOne, BugCrowd) leverage crowd-sourced vulnerability discovery to improve continuous coverage, reduce the cost of discovery, and serve as an integral complement to internal red teams. With the rise of AI-generated bug reports, little work exists to help bug hunters understand why these reports are labeled as invalid. To improve report quality and reduce reviewers' burden, it is critical to predict invalid reports and interpret invalid reasons.

In this work, we conduct an empirical study with the purpose of helping bug hunters understand the validity of reports. We collect a dataset of 9,942 disclosed bug bounty reports, including 1,400 invalid reports, and evaluate whether state-of-the-art large language models can identify invalid reports. While models such as GPT-5, DeepSeek, and a fine-tuned RoBERTa achieve strong overall accuracy, they consistently struggle to detect invalid cases, showing a tendency to over-accept reports. To improve invalidity detection, we build a taxonomy of rejection reasons for Information Disclosure vulnerabilities and incorporate it into a retrieval-augmented generation (RAG) framework. This approach substantially improves classification consistency and reduces bias. We also examine whether reviewer decisions may be influenced by factors beyond the content of the report. Our analysis shows that reporters with higher reputations tend to receive more favorable outcomes in borderline cases, suggesting that perceived expertise can influence review judgments.

Overall, our findings highlight the challenges of invalid report identification and show that combining LLMs with structured reviewer knowledge can support more transparent and consistent vulnerability report review.

Index Terms—Bug-bounty programs, Invalid vulnerability reports, LLMs, Security

I. INTRODUCTION

“Given enough eyeballs, all bugs are shallow.” This well-known principle, often referred to as Linus’ Law, has been empirically supported by analyses of bug bounty programs [2]. Bug bounty programs (e.g., HackerOne, BugCrowd) leverage crowd-sourced vulnerability discovery to improve continuous coverage, reduce the cost of discovery, and serve as an integral complement to internal red teams. However, as vulnerability discovery activities scale up, there is a growing expectation gap between bug hunters and reviewers (i.e., vendor staff). Prior studies have shown that reviewing bug bounty submissions is highly resource-intensive and time-consuming for organizations [3]. There is an increasing number

[†]Corresponding author.

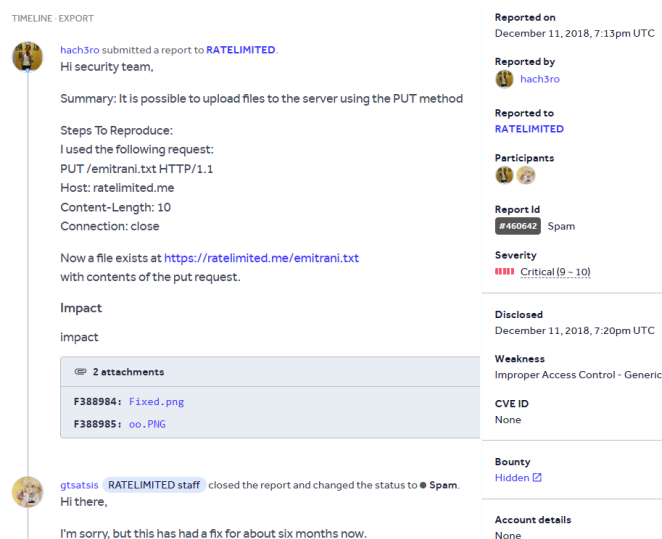


Fig. 1: Example invalid bug bounty report (#460642, HackerOne [1]). The submission, titled “HTTP PUT method enabled,” was closed as *Spam* because it reused a previously disclosed demo.

of spam or AI-generated reports that pollute the overall bug bounty ecosystem and make it harder to identify genuine vulnerabilities. From the reviewers’ perspective, these programs are increasingly challenged by a surge of invalid or low-quality submissions. Curl project founder Daniel Stenberg, for instance, publicly expressed frustration at the overwhelming number of AI-generated bug reports that waste valuable reviewing efforts and slow down the overall vulnerability handling process [4].

To the best of our knowledge, limited studies have investigated how to help hunters predict and understand the invalidity of their reports before submission. The closest work is by Shafiq et al. [5], but they focus only on out-of-scope invalid reports, without addressing other types of invalidity, and they do not analyze valid submissions. Moreover, few studies have examined whether modern large language models (LLMs) can effectively support reviewers in making fair and consistent decisions. With their strong capacity for textual reasoning and contextual understanding, LLMs present a promising avenue for modeling how reviewers assess report validity.

To shed light on the invalidity reasons, we conduct an empirical study using all the publicly disclosed vulnerability reports from HackerOne, one of the leading security bug bounty platforms, where vendors pay cash to reward third-party security researchers' efforts in helping them find bugs. The security bugs covered include web, mobile, cloud, IoT, etc. We collect 9,942 vulnerability reports from HackerOne (8,542 valid and 1,400 invalid).

We explore the following research questions:

RQ1: How effective are LLMs in predicting invalidity?

We first benchmark state-of-the-art LLMs, including GPT-5 and DeepSeek, alongside a fine-tuned RoBERTa model to evaluate their baseline ability to classify bug bounty reports as valid or invalid.

RQ2: Does incorporating reviewer knowledge help predict invalidity?

Next, we investigate incorporating three aspects of reviewer knowledge: vendor scope definitions, program policies, and a taxonomy of invalid report types. We integrate the taxonomy using a retrieval-augmented generation (RAG) framework to models.

RQ3: Does bias exist on reviewers' invalidity judgment?

Finally, we conduct a fairness analysis to examine whether reviewer decisions are influenced by non-content factors such as reporter reputation. We identify pairs of reports within the same weakness category that are semantically similar but have different validation statuses. By comparing these pairs, we analyze the extent to which reputation differences correlate with outcome discrepancies.

Our findings for **RQ1–RQ3** are summarized as follows:

- 1) LLMs such as GPT-5, DeepSeek, and fine-tuned RoBERTa model show strong overall accuracy but struggle to identify invalid reports, revealing a bias toward acceptance.
- 2) Adding reviewer knowledge in the form of a taxonomy of invalid report types helps the models detect invalid cases more reliably and make decisions more consistently.
- 3) Review outcomes are not determined solely by the content itself. High-reputation reporters receive more favorable outcomes even for reports that are nearly identical to those from low-reputation reporters.

Together, these results provide new insights into the human factors and reasoning cues behind bug bounty validity decisions.

II. BACKGROUND AND MOTIVATION

A. How Bug Bounty Programs Work

Overview of Bug Bounty Platforms Bug bounty platforms are gaining widespread attention across industry, government, and academia [6]–[8]. A bug bounty program is a structured vulnerability disclosure mechanism that allows independent security researchers to report security flaws to organizations in

exchange for monetary rewards or recognition. As described by HackerOne [9], companies define eligible assets and vulnerability types (*scope*), and offer bounties based on the severity and impact of verified findings.

Workflow of Bug Bounty Process. The process typically involves four stages: (1) researchers discover and submit vulnerabilities, (2) reviewers validate the reports and determine their legitimacy, (3) vendors fix confirmed issues, and (4) rewards are issued according to program policy. Reports that are out of scope, invalid, or have negligible impact are rejected without reward.

This workflow enables organizations to continuously test and improve their security through crowdsourced expertise while relying on consistent review and reward mechanisms managed by the platform.

Major Bug Bounty Platforms. Bug bounty platforms such as HackerOne and Bugcrowd mainly host vendor-specific programs, where companies define their own scopes and review reports internally. In contrast, Huntr [10] focuses on vulnerabilities in open-source projects: researchers still submit reports through the platform, but the issues are routed to project maintainers (e.g., via GitHub). Prior studies have investigated multiple dimensions of bug bounty platforms.

B. Challenges in AI-driven Bug Hunting Era

Recent years have brought a surge of AI-generated submissions in bug bounty programs. Many of these reports appear convincing at first glance but contain fabricated evidence or shallow reasoning, creating significant overhead for triage teams. High-profile maintainers describe being “effectively DDoS’ed” by low-effort, machine-written vulnerability claims, prompting them to adopt pre-screening mechanisms and stricter submission filters to manage the flood of incoming reports [4].

Multiple public accounts have documented large waves of such “AI-slop” reports across both open-source projects and commercial bounty platforms, where maintainers spend hours disproving technically invalid claims that nonetheless sound credible [11]–[13]. At the same time, AI tools are increasingly used by hunters to explore new attack surfaces and automate vulnerability discovery [14], [15]. While this accelerates legitimate findings, it also increases the total volume of submissions that reviewers must process [16]. As language models continue to improve, surface-level cues will no longer suffice to identify unreliable reports, forcing reviewers to rely more on evidence verification—such as reproducing proof-of-concept code or confirming exploit chains—and highlighting the need for semi-automated triage support [15].

Beyond AI-generated noise, reviewers continue to face familiar challenges that have persisted for years. Differences in security expertise among maintainers often lead to inconsistent judgments on subtle exploitation details [17]. Meanwhile, platforms still receive large numbers of duplicate or near-duplicate reports describing the same underlying issue, adding to the workload and slowing response times [7], [18], [19]. While such challenges are not new, the growing number of

submissions aided by AI tools has made them more prominent and harder to handle.

Implications. The largely manual nature of bug bounty review, combined with the rise of low-quality and AI-generated submissions, highlights the need for more consistent and explainable validity judgments. Motivated by this need, our study investigates whether LLMs—when guided by structured reviewer knowledge—can help improve the reliability of the review process.

III. DATASET

A. Data Collection

We collect a total of 9,942 disclosed vulnerability reports from the HackerOne platform. Among these reports, 8,542 were accepted and 1,400 were not accepted. The dataset is randomly divided into training and test subsets at a 4:1 ratio while preserving the original distribution of valid and invalid reports, resulting in 7,953 training cases and 1,989 test cases. Each report contains multiple metadata fields describing its submission details, review history, and reporter profile. Figure 1 shows an example report entry. To facilitate analysis, we extract the following fields:

- **Basic Information:** Includes the report title, submission date, reporter name, target program (Reported to), and final disclosure date.
- **Vulnerability Attributes:** Includes the reported Weakness category, CVE ID, and Severity.
- **Review Timeline:** Records all reviewer–hunter interactions, including comments, status changes, and disclosure events.
- **Reporter Profile:** Extracted from the reporter’s public profile section, including long-term performance metrics such as Signal, Impact, and Reputation.
- **Final Status:** Indicates the triage outcome assigned by reviewers (*Resolved*, *Informative*, *Not Applicable*, *Spam*, etc.). Reports marked as *Resolved* are treated as valid and fixed vulnerabilities, while those labeled *Not Applicable* or *Spam* are considered invalid. The *Informative* status refers to submissions that contain useful information but do not require any security action [20].

B. Data Distribution Analysis

Vulnerability Types. Table I lists the top ten reported weaknesses. While the table presents disaggregated data, we observe that multiple variants of Cross-Site Scripting (XSS) together account for 13.8% of all submissions. Information Disclosure remains the single most frequent weakness (9.4%), followed by Access Control and Authentication issues, which together represent more than 10% of the dataset.

Target Organizations. Table II lists the top programs. Government and defense (U.S. DoD 8.3%) and open-source initiatives (Internet Bug Bounty 6.8%) are heavily targeted.

TABLE I: Top 10 Vulnerability Types in HackerOne Reports

Rank	Weakness Type	Count	%
1	Information Disclosure	931	9.4
2	Improper Access Control - Generic	612	6.2
3	Cross-site Scripting (XSS) - Reflected	483	4.9
4	Violation of Secure Design Principles	461	4.6
5	Cross-site Scripting (XSS) - Stored	451	4.5
6	Improper Authentication - Generic	442	4.4
7	Cross-site Scripting (XSS) - Generic	436	4.4
8	Uncontrolled Resource Consumption	414	4.2
9	Business Logic Errors	306	3.1
10	Cross-Site Request Forgery (CSRF)	304	3.1

TABLE II: Top 10 Target Programs In HackerOne report

Rank	Organization	Reports	%
1	U.S. Dept of Defense	828	8.3
2	Internet Bug Bounty	677	6.8
3	Nextcloud	504	5.1
4	HackerOne	384	3.9
5	Shopify	339	3.4
6	Node.js Modules	303	3.0
7	GitLab	250	2.5
8	curl	180	1.8
9	X / xAI	163	1.6
10	VK.com	162	1.6

IV. RQ1: HOW EFFECTIVE ARE LLMs IN PREDICTING INVALIDITY?

A. Motivation

Although LLMs such as GPT-5 and DeepSeek have demonstrated strong reasoning ability across many domains, it remains unclear whether they can generalize to the nuanced review process of bug bounty platforms, where rejection decisions often depend on implicit reviewer knowledge. Understanding their baseline capability is crucial before introducing any reviewer-specific context or knowledge augmentation.

B. Methodology

We first examine the capability of state-of-the-art large language models to review bug bounty reports. Specifically, we evaluate GPT-5 and DeepSeek on the task of classifying HackerOne reports as valid or invalid. Each model is prompted to act as an expert bug bounty reviewer. The LLMs are zero-shot prompted using the following instruction:

You are an expert bug bounty reviewer.
Given the following HackerOne report, classify it strictly as:
1 = valid (valid vulnerability report), 0 = invalid (Informative, Not Applicable, Spam).
Report to evaluate:
[report_text]
Output only:
1 = VALID, 0 = INVALID

To establish a supervised baseline, we fine-tune a RoBERTa model on the collected HackerOne reports. The model is trained with a learning rate of 2×10^{-5} , weight decay of 0.01, and 3 epochs.

C. Experimental Results and Analysis

Metric Definition. We treat **valid reports** as the positive class (P) and **invalid reports** as the negative class (N). Accordingly:

TP : valid reports correctly predicted as valid,
 FP : invalid reports incorrectly predicted as valid,
 FN : valid reports incorrectly predicted as invalid,
 TN : invalid reports correctly predicted as invalid.

- **Accuracy (Acc):** $(TP + TN) / (TP + FP + FN + TN)$.
- **Recall_v / Recall_{inv}:** $TP / (TP + FN)$ for valid class, $TN / (TN + FP)$ for invalid class.
- **Precision_v / Precision_{inv}:** $TP / (TP + FP)$ for valid class, $TN / (TN + FN)$ for invalid class.
- **F1_v / F1_{inv}:** Harmonic mean of precision and recall for each class. $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.
- **Macro-F1:** Average of F1_v and F1_{inv}, representing overall balance between the two classes.

GPT-5 and Deepseek. As shown in Table III, both large language models achieve high overall accuracy (GPT-5: 0.82; DeepSeek: 0.79), yet exhibit a strong bias toward predicting valid reports. GPT-5 reaches a high recall on valid cases (0.91) but extremely low recall on invalid ones (0.28), indicating a false negative rate of roughly 72%, i.e., most invalid reports are mistakenly accepted. DeepSeek alleviates this bias slightly (invalid recall = 0.44) while maintaining comparable precision, suggesting improved discrimination between valid and invalid classes. However, both models' F1 scores for invalid reports (0.30–0.37) remain far lower than for valid ones (0.87–0.90), indicating that overall accuracy can mask substantial disparities in per-class performance.

Fine-tuned RoBERTa. Because LLMs show limited performance, we are interested in examining whether a fine-tuned RoBERTa can perform better. The fine-tuned RoBERTa baseline achieves a better balanced performance (Macro-F1 = 0.72). Although its recall on invalid reports (0.43) is comparable to DeepSeek's, it retains both strong recall (0.96) and precision (0.93) for valid cases, resulting in higher reliability overall. RoBERTa also improves F1 on invalid reports to 0.50, suggesting that task-specific fine-tuning enables better recognition of linguistic and structural patterns characteristic of invalid submissions. Nevertheless, the model still fails to correctly handle borderline or unclear reports, showing that imbalanced data and implicit reviewer cues remain major challenges for automated review.

Most misclassifications come from two situations: (1) out-of-scope submissions that look plausible as a vulnerability but lack scope knowledge, and (2) reports that lack sufficient textual detail. Some reports rely mainly on screenshots or video

attachments, which provide too little textual content for an LLM to reason about. For example, Report #226514 [21] contains only a short description and a PNG image showing a full-path disclosure on `airship.paragonie.com`. Although the issue is real, the reviewer marked it as "Informative" because the affected asset was out of scope. Such cases motivate our use of vendor-scope information and retrieved reference examples in RQ2, which help the model better handle borderline cases that lack enough text or depend on contextual program rules.

D. Summary of Findings

Our results show that large language models can handle bug bounty report validation fairly well in general, but they still struggle to identify invalid cases and often accept reports that only look like real vulnerabilities. Fine-tuning a task-specific model such as RoBERTa helps reduce this bias and makes the predictions more balanced between valid and invalid reports. However, the persistent performance gap across classes suggests that the models rely mainly on surface-level linguistic patterns rather than deeper reviewer reasoning.

V. RQ2: DOES INCORPORATING REVIEWER KNOWLEDGE HELP PREDICT INVALIDITY?

A. Motivation

The baseline model from RQ1 tends to label most reports as valid, showing a bias toward acceptance. We hypothesize that this bias stems from the model's lack of reviewer-level contextual knowledge, such as what kinds of reports are typically rejected and why. To address this gap, we investigate whether providing explicit reviewer knowledge can help LLMs make more consistent and accurate decisions when identifying invalid reports.

B. Methodology

We enrich the model input with two forms of structured context: (1) vendor scope, which specifies the coverage of assets that are considered valid for submission in each program, and (2) a taxonomy of invalid report types. As a preliminary study, we focus on the *Information Disclosure* category (Table IV). We choose *Information Disclosure* because it appears most frequently in our dataset (9.4%) and covers diverse report forms. Its prevalence and variability make it a suitable category for an initial analysis of reviewer consistency. The underlying methodology is not specific to this category and can be transferred to other vulnerability classes, particularly those where reviewers rely heavily on written explanations rather than executable proofs of concept.

Vendor Scope Extraction. During inference, we use the vendor name in the report's "Reported to" field to fetch the corresponding policy scope page and extract the raw scope entries. Figure 2 shows an example of the vendor scope table for the Palo Alto Software program on HackerOne. Each entry lists an *asset name*, its *type*, and the *coverage status* indicating whether the asset is *In Scope* or *Out of Scope*. Additional metadata, such as severity limits and bounty eligibility, are available but not used in our model. We

TABLE III: Models' performance on the test set and entire dataset (Valid treated as positive, Invalid as negative).

Model	Acc	Rec _{val}	Rec _{inval}	Pre _{val}	Pre _{inval}	F1 _{val}	F1 _{inval}	Mac-F1
GPT-5	0.822	0.912	0.275	0.885	0.338	0.898	0.303	0.601
DeepSeek	0.789	0.846	0.439	0.902	0.319	0.873	0.369	0.621
RoBERTa (finetuned)	0.881	0.956	0.425	0.910	0.610	0.932	0.501	0.717
GPT-5 (All)	0.818	0.914	0.324	0.882	0.355	0.897	0.339	0.618
DeepSeek (All)	0.795	0.853	0.465	0.899	0.352	0.875	0.403	0.639

TABLE IV: Taxonomy of rejection reasons for invalid HackerOne reports.

Status	Category	Description
Informative	Technical Issues	Known bugs, false positives, or unreproducible problems that do not constitute security vulnerabilities.
	Third-party Related Issues	Issues caused by external dependencies or third-party services outside program control.
	Scope/Policy Issues	Reports outside the defined program scope or violating disclosure policies.
	Risk Assessment	Valid findings assessed as low impact or below the action threshold.
	Special Scenarios	Non-standard cases such as debug modes, joke submissions, or physical access requirements.
	Path Disclosure Related	Exposure of file paths or directories without security impact.
	Version Disclosure Related	Disclosure of version or build information that is public or non-sensitive.
	User/Data Exposure	Non-sensitive user or general data exposed intentionally or without risk.
	File/Directory Related	Public or properly access-controlled files misreported as sensitive.
	Configuration/Credentials Related	Test credentials or placeholder configurations without real security implications.
Not Applicable	System Function / Security Trade-off Design	Intended behavior or accepted design trade-offs balancing usability and security.
	Tool/Protocol Behavior	Expected behavior of underlying tools, frameworks, or infrastructure.
Not Applicable	Not Applicable	The report doesn't contain a valid reproducible issue, and the security implications have not been demonstrated.
Duplicate	Duplicate	Issues already reported or resolved under previous submissions.
Spam	Spam / Irrelevant	Reports without legitimate security relevance, incomprehensible content, or promotional material.

focus instead on coverage information, which defines valid submission boundaries.

To incorporate vendor policies into the model input, we insert the following vendor-scope instructions into the *[vendor_scope]* placeholder of the main evaluation prompt:

Legend:

- **Asset name:** Asset or endpoint name.
- **Type:** Asset type such as Domain, Source code, or Wildcard.
- **Coverage:** Whether this asset is in program scope (True = In Scope, False = Out of Scope).

- Asset name: [asset name 1] | Type: [asset type 1] | Coverage: [True or False]

Taxonomy RAG. To enhance the reasoning of the model with the knowledge of the reviewer, we design a retrieval-augmented generation (RAG) framework that integrates real examples of previously reviewed reports. The intuition is that

by exposing the LLM to concrete invalid and valid cases with known outcomes, it can reason by analogy rather than relying purely on superficial text features, thereby reducing hallucination.

We first build a reference knowledge base from manually annotated *Information Disclosure* reports. Each taxonomy type includes: (1) A short textual description of the invalid reasons, and (2) Several representative invalid reports that exemplify this reason.

For each test report belonging to the *Information Disclosure* category, we retrieve the top- k most semantically similar reports ($k = 3$) from the knowledge base using cosine similarity over text embeddings. We embed each report using the `sentence-transformers/all-mpnet-base-v2` model from the SentenceTransformer library and use normalized embeddings to compute similarity. We adopt a similarity threshold of 0.8. This value falls within the range commonly used to identify near-duplicate or paraphrastic sentences in embedding-based similarity work, where higher cosine scores are shown to reflect strong semantic closeness [23]. To check

Asset name ↑	Type ↑	Coverage ↑	Max. severity ↓	Bounty ↑
app.liveplan.com	Domain	In scope	Critical	\$ Ineligible
www.paloalto.com	Domain	In scope	Medium	\$ Ineligible
www.liveplan.com	Domain	In scope	Medium	\$ Ineligible
www.bplans.com	Domain	In scope	Low	\$ Ineligible
www.teamoutpost.com	Domain	Out of scope	None	\$ Ineligible
www.outpost.co	Domain	Out of scope	None	\$ Ineligible
www.mplans.com	Domain	Out of scope	None	\$ Ineligible

Fig. 2: Example of vendor scope entries on HackerOne (Palo Alto Software program) [22]. Each entry defines the in-scope and out-of-scope assets for valid submissions.

the validity of this threshold, we manually review a sample of similarity-based pairs within the *Information Disclosure* category and confirm that it captures reports describing essentially the same issue. To maintain contextual consistency, retrieval is restricted to reports under the same weakness label, excluding unrelated or duplicate entries. For each retrieved report, we include its title and full content as contextual evidence. If the retrieved report is invalid, we additionally add its taxonomy type and corresponding description. This structured retrieval allows LLMs to recognize patterns of invalid reasoning and to ground its decision on actual reviewer judgments rather than surface-level keywords. During inference, GPT-5 receives the vendor’s in-scope information (if available), the retrieved reference reports, and the new report to be evaluated. The model is then instructed to classify the report as either **VALID (1)** or **INVALID (0)** based on all provided information.

The main evaluation prompt we use follows the instructions below:

You are an expert bug bounty assistant.
Vendor’s in-scope assets and policies: *[vendor_scope]*
Below are example reports retrieved from the knowledge base that are semantically similar to the current report. Each example includes the report content, type, and description.
Use them as reasoning references — do not classify them again.
Retrieved reference reports:
[rag_reports (+type, +desc) if invalid]
Now, determine whether the following new report is **VALID (1)** or **INVALID (0)**.
Report to evaluate: *[report_text]*
Output only:
1 = VALID, 0 = INVALID

C. Experimental Results and Analysis

We compare four settings on the same subset: (i) the GPT-5 baseline (report only), (ii) **+Scope** (vendor scope added), (iii) **+Taxonomy RAG**, and (iv) **+Taxonomy RAG + Scope**. The baseline achieves high accuracy on valid cases but low recall on invalid ones, confirming a strong acceptance bias.

Adding **vendor scope** alone provides limited improvement and, in some cases, slightly reduces accuracy. This is likely because the vendor scope descriptions are often generic or high-level (e.g., program URLs or IP ranges) and provide little semantic guidance for distinguishing invalid content.

In contrast, the **taxonomy-based RAG** approach substantially enhances invalid recall and overall consistency. By retrieving semantically related examples with explicit rejection reasons, the model learns to reason from prior invalid patterns instead of relying purely on surface similarity. As shown in Table V, this method yields the best overall performance, achieving an accuracy of 0.885 and a Macro-F1 of 0.815.

For **GPT-5 + Taxonomy RAG + Vendor Scope**, the performance did not continue to improve. Instead, a slight drop was observed compared with using taxonomy RAG alone (Macro-F1 = 0.725). This suggests that combining multiple forms of external context does not necessarily yield additive benefits, and the interaction between different types of contextual signals may require further investigation.

Case Study of RAG. We examine a representative case to show how RAG complements missing contextual information.

Report #2215434 describes an information disclosure issue where a PoC video unintentionally reveals a hacker’s email address [24]. Because the textual description is brief and the LLM cannot interpret video content, the baseline GPT-5 model incorrectly labeled it as invalid. With the RAG method, the model retrieved Report #2134874 as the most similar example, which is the exact report cited inside #2215434. The retrieved report provides a detailed description of the same vulnerability type (emails exposed in a PoC video), which compensates for the missing context and helps GPT-5 recognize that #2215434 indeed demonstrates a valid privacy disclosure. Table VI summarizes this case.

D. Summary of Findings

Our results show that adding reviewer knowledge can substantially improve the model’s ability to recognize invalid reports. The taxonomy-based RAG approach provides concrete examples and reasoning cues, which help the model make more consistent judgments and reduce its bias toward accepting weak submissions. In contrast, vendor scope information alone brings little benefit because it is often too general to guide the classification process. Overall, structured knowledge that reflects how reviewers actually reason about invalidity proves more useful than static metadata when improving LLM-based bug bounty validation.

TABLE V: Performance comparison on the *Information Disclosure* subset. Valid is treated as positive and Invalid as negative.

Setting	Acc	Rec _{val}	Rec _{inval}	Pre _{val}	Pre _{inval}	F1 _{val}	F1 _{inval}	Mac-F1
GPT-5	0.781	0.872	0.293	0.868	0.299	0.870	0.296	0.583
+Scope	0.699	0.768	0.333	0.861	0.211	0.812	0.259	0.535
+Tax-RAG	0.885	0.891	0.856	0.971	0.592	0.929	0.700	0.815
+Tax-RAG+Scope	0.803	0.791	0.863	0.969	0.434	0.871	0.578	0.725

TABLE VI: Case study: how RAG retrieved a related report that helped correct GPT-5’s judgment.

Item	Description
Target report	#2215434: PoC video exposes a hacker’s email address in a local folder.
Baseline prediction	Misclassified as invalid due to limited textual detail.
Retrieved reports	#2134874: Describes the same exposure pattern (emails visible in PoC video), referenced inside #2215434.
Improvement	The retrieved report supplies explicit textual evidence of the same vulnerability type, clarifying the missing context.
Outcome	Model correctly revised decision, classifying #2215434 as a valid information disclosure.

VI. RQ3: DOES BIAS EXIST ON REVIEWERS’ INVALIDITY JUDGMENT?

A. Motivation

While the previous experiments mainly focused on textual content, we also explore whether non-content factors might influence reviewers’ final judgments. In practice, report validation is not a purely mechanical process. It often depends on how the report is written, who submitted it, and how reviewers perceive its credibility. We investigate whether reports with similar content could still receive different outcomes.

B. Methodology and Experiments

Building on the similarity computation used in RQ2, we identify the top- k semantically closest reports within each weakness category to form comparable pairs. We then remove pairs that share the same status or contain duplicates, retaining only those with a similarity score above 0.8. This filtering ensures that the compared pairs describe essentially the same vulnerability but were evaluated differently during review.

Next, we align each pair’s review outcomes with the corresponding reporters’ reputation scores from HackerOne. We assign ordinal values to report statuses according to their impact on reputation¹ [20], and treat missing reputation scores as 0. This enables us to examine whether a “better” outcome

¹Following the ranking of HackerOne’s closed report states: *Resolved* > *Informative* > *Not Applicable* > *Spam*, which correspond to reputation changes of +7, 0, -5, and -10 respectively.

(i.e., a higher-priority status) tends to correspond to reporters with a higher reputation.

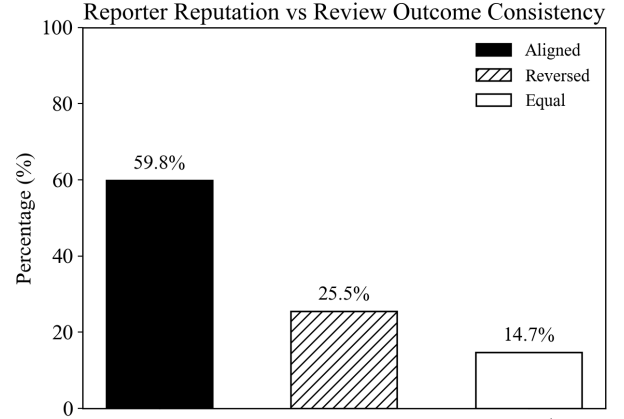


Fig. 3: **Aligned**: the higher-reputation reporter received a higher-ranked status (e.g., Resolved over Informative); **Equal**: both had identical reputation; **Reversed**: the higher-reputation reporter received a lower-ranked status.

Inconsistency Distribution Analysis. As shown in Figure 3, we analyze a total of 259 *report-neighbor pairs*, each consisting of two semantically similar reports identified within the same weakness category using cosine similarity of their textual embeddings (similarity ≥ 0.8). Across these pairs, we find that in 155 cases (59.85%), the report with the *higher status outcome* also came from a reporter with a higher reputation. Among the remaining 104 pairs (40.15%), 14.67% cases had identical reputation scores between the two reporters, while 25.48% cases showed the opposite trend, where the report with a higher-ranked outcome was submitted by a lower-reputation reporter. Although this is not an overwhelming majority, the distribution still shows a clear correlation between a reporter’s reputation and the perceived validity of their submissions. The results suggest that reputation or experience may influence how reviewers interpret and evaluate technically similar reports.

This pattern can also be visually confirmed in Figure 3, where the *Aligned* bar dominates the distribution, indicating that reviewers tend to favor experienced reporters when evaluating similar vulnerability reports.

Fairness Test Near the Decision Boundary. To evaluate whether reporter reputation influences reviewer leniency, we perform a fairness analysis on *borderline* reports—those that the model predicts as invalid with high confidence but are

TABLE VII: Examples of similar reports with inconsistent reviewer outcomes.

Attribute	Case 1	Case 2
Example Clickjacking — Different Programs		
Report ID	#688546 (Palo Alto Software) [25]	#832593 (Kubernetes) [26]
Reporter	paramdham (reputation = 8,342)	hackerboy404 (reputation = 44)
Target	app.outpost.co	kubernetes.io
Vulnerability Type	Clickjacking (missing X-Frame-Options)	Clickjacking (same issue and PoC)
Proof of Concept	Short iframe-based PoC	Similar iframe PoC reused
Reviewer Decision	<i>Resolved</i> — vendor acknowledged and fixed the issue	<i>Informative</i> — no sensitive state change involved
Outcome	Accepted and rewarded	Rejected as low impact

TABLE VIII: Fairness evaluation across reputation groups. $val|inval$ denotes the percentage of human-accepted reports among model-rejected ones, and $inval|val$ denotes the percentage of human-rejected reports among model-accepted ones.

Group	TPR	FPR	FNR	$val inval(\%)$	$inval val(\%)$
High reputation	0.965	0.694	0.035	59.26	5.31
Low reputation	0.944	0.534	0.056	31.21	13.01
Δ (High-Low)	+0.021	+0.161	-0.021	+28.05	-7.70

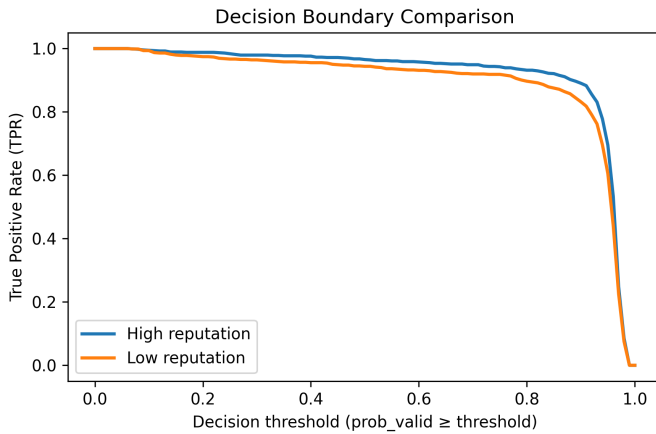


Fig. 4: Decision boundary comparison between high and low reputation reporters.

labeled as valid by human reviewers.

Here we use the fine-tuned RoBERTa model (the best-performing system in RQ1) as a reference to locate these borderline reports. We select reports satisfying $P(\text{invalid}) \geq 0.7$ and label = 1 (valid in ground truth), and then divide them into two groups by reporter reputation (high vs. low, split by the median reputation score).

Following prior fairness studies [27], [28], we measure potential bias using standard classification metrics:

- **True Positive Rate (TPR)** — proportion of valid reports correctly accepted.
- **False Positive Rate (FPR)** — proportion of invalid-like reports incorrectly accepted.
- **False Negative Rate (FNR)** — proportion of valid-like

reports incorrectly rejected.

- $val | inval$ — percentage of human-accepted reports among model-rejected ones:

$$val | inval = \frac{\#\{\text{model predicts 0, human label} = 1\}}{\#\{\text{model predicts 0}\}}.$$

- $inval | val$ — percentage of human-rejected reports among model-accepted ones:

$$inval | val = \frac{\#\{\text{model predicts 1, human label} = 0\}}{\#\{\text{model predicts 1}\}}.$$

A fair review process should yield similar TPR and FPR across reputation groups. As shown in Table VIII, the TPR values for the two groups are close to each other ($\Delta\text{TPR} = 0.02$), meaning both groups have a similar rate of correctly accepted valid reports. In contrast, the FPR gap is much larger ($\Delta\text{FPR} = 0.16$), which suggests that high-reputation reporters are more likely to have invalid-like reports accepted. The other two metrics show the same trend: high-reputation reporters have a much higher $val | inval$ score (+28.05%), indicating that model-rejected reports from this group are more often accepted by human reviewers. They also have a lower $inval | val$ score (−7.70%), meaning their accepted reports are less likely to be judged invalid by humans. Taken together, these results show that reviewers tend to be more lenient toward high-reputation reporters.

This aligns with the intuition that human reviewers apply a more lenient decision boundary to trusted reporters. The same pattern is reflected in Figure 4, where the TPR curve for high-reputation reporters consistently lies above that of low-reputation ones.

Example (Clickjacking, Palo Alto Software vs. Kubernetes). From Table VII, we can see that the example compares two reports describing the same clickjacking vulnerability caused by missing X-Frame-Options headers. Despite nearly identical proof-of-concept code, the results are different: the report submitted by a higher-reputation hunter (paramdham) was accepted and fixed, while the one by a lower-reputation hunter (hackerboy404) was marked as *Informative*.

C. Summary of Findings

Our analysis provides preliminary evidence that bug bounty reviews may not always be entirely objective. Reports with

nearly identical content can still receive different outcomes, and higher-reputation reporters tend to obtain slightly more favorable results. This suggests that reviewer perception, shaped by a reporter's experience or credibility, may influence decisions in borderline cases. Understanding this tendency can help future AI-assisted review systems make the process fairer and more consistent.

VII. RELATED WORK

A. Empirical Studies on Bug Bounty Platforms

Existing empirical studies have examined bug bounty platforms from various perspectives, including economic efficiency [29]–[32], participant motivation [33], [34], and maintainers' security practices [35]–[37], while overlooking the review process itself.

Finifter et al. [38] conducted one of the earliest quantitative analyses of vulnerability reward programs, finding that vendor-sponsored bounties were cost-effective compared to hiring additional security staff. Akgul et al. [33] and Barre et al. [34] investigated hunters' motivations, highlighting the role of financial incentives, reputation, and learning opportunities, while also identifying demotivating factors such as delayed or unfair evaluations. Ayala et al. [37] examined open-source maintainers on Huntr and observed that reviewers' acceptance decisions were influenced by report quality and the presence of proof-of-concept (PoC) exploits rather than by formalized criteria. However, these studies do not analyze how reviewers actually determine validity, what criteria they apply, or how consistent their decisions are. Our work contributes to filling this gap by analyzing how reviewers make validity decisions and what factors correlate with different outcomes.

In addition, from a methodological standpoint, most prior empirical studies on bug bounty platforms have been conducted at a relatively limited scale. For example, Maillart et al. [2] analyzed activity patterns across only 35 public programs on HackerOne, illustrating how vulnerability discoveries decline over time after program launch. Similarly, Shafigh et al. [5] examined just 1,028 fully disclosed invalid reports to develop an out-of-scope taxonomy. In contrast, our work utilizes a much larger dataset of 9,942 disclosed reports across 316 programs, which allows for a more comprehensive and statistically grounded analysis of reviewer decisions and invalidity patterns.

B. LLM-Assisted Bug Bounty Analysis

Research on applying LLMs or agent-based systems to bug bounty review remains scarce. Existing work has primarily focused on using LLMs to support hunters in vulnerability discovery and exploit automation. For instance, *The Hacker's Guide to LLMs* [39] explores how LLMs can be used for attack simulation, prompt-based exploitation, and adversarial testing within bug bounty workflows. However, these studies address the offensive and discovery aspects of bug bounty processes rather than the review and validation stage.

Our work investigates the use of LLMs as review agents that leverage natural language reasoning and reviewer knowledge (such as report taxonomies and program scope) to analyze

report validity. To the best of our knowledge, no prior empirical study has systematically modeled human reviewer decisions using LLMs or agent-based reasoning in the context of bug bounty report validation.

VIII. CONCLUSION

This paper studied how bug bounty reviewers decide the validity of vulnerability reports and explored whether LLMs can assist in this process. After collecting 9,942 disclosed HackerOne reports, we evaluated GPT-5, DeepSeek, and a fine-tuned RoBERTa model on report validity prediction. Although the models performed well on valid reports, they often failed to recognize invalid ones, showing a clear tendency toward acceptance. Adding reviewer knowledge through a taxonomy-based retrieval method improved both recall and consistency, suggesting that structured contextual cues help models make more balanced judgments. Our analysis of similar reports with different outcomes also indicates that reviewer perception and reporter reputation may influence validation results.

IX. LIMITATIONS AND FUTURE WORK

Although our study provides new empirical insights, it has several limitations.

First, our current taxonomy focuses only on the *Information Disclosure* category, which limits the generality of our conclusions. We plan to expand it to cover other major vulnerability types, including cross-site scripting, access control, and authentication issues, to build a more comprehensive reviewer knowledge base.

Furthermore, our dataset includes only publicly disclosed reports on HackerOne; private programs may exhibit different review dynamics. We will extend our dataset to include reports from other bug bounty platforms such as Bugcrowd and Huntr, as well as private disclosure programs. A larger and more diverse dataset will allow for a broader analysis of reviewer behavior and improve model generalization across platforms.

In addition, our evaluation relies on text-based similarity and may overlook non-textual cues such as proof-of-concept videos or attachments. These aspects will be addressed in our future work.

Through these efforts, we aim to better understand how reviewer knowledge and cross-platform data can enhance the fairness and consistency of bug bounty validation.

X. ETHICAL CONSIDERATIONS

All data used in this study come from publicly disclosed HackerOne reports. We analyze only information that is voluntarily published by reporters and vendors, and we do not attempt to identify individuals or make judgments about specific reviewers. All results are presented in aggregate, and examples are used solely for scientific illustration. Because no private data or human intervention was involved, this work is exempt from institutional review board (IRB) oversight under standard guidelines.

REFERENCES

- [1] hach3ro. (2018, December) Http put method enabled. RATELIMITED via HackerOne. Status: Spam; Weakness: Improper Access Control - Generic; Severity: Critical (9–10); Disclosed: Dec 11, 2018. [Online]. Available: <https://hackerone.com/reports/460642>
- [2] T. Maillart, M. Zhao, J. Grossklags, and J. Chuang, “Given enough eyeballs, all bugs are shallow? revisiting eric raymond with bug bounty programs,” *Journal of Cybersecurity*, vol. 3, no. 2, pp. 81–90, 2017.
- [3] T. Walshe and A. C. Simpson, “Coordinated vulnerability disclosure programme effectiveness: Issues and recommendations,” *Computers & Security*, vol. 123, p. 102936, 2022.
- [4] C. Jones, “Curl project founder snaps over deluge of time-sucking ai slop bug reports,” *The Register*, May 2025. [Online]. Available: https://www.theregister.com/2025/05/07/curl_ai_bug_reports/
- [5] S. Shafiqh, B. Benattallah, C. Rodríguez, and M. Al-Banna, “Why some bug-bounty vulnerability reports are invalid? study of bug-bounty reports and developing an out-of-scope taxonomy model,” in *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2021, pp. 1–6.
- [6] R. Böhme, “A comparison of market approaches to software vulnerability disclosure,” in *International conference on emerging trends in information and communication security*. Springer, 2006, pp. 298–311.
- [7] A. Laszka, M. Zhao, and J. Grossklags, “Banishing misaligned incentives for validating reports in bug-bounty platforms,” in *European Symposium on Research in Computer Security*. Springer, 2016, pp. 161–178.
- [8] A. Ozment, “Bug auctions: Vulnerability markets reconsidered,” in *Third workshop on the economics of information security*, 2004, pp. 19–26.
- [9] H. Team. (2024, March) What are bug bounties and how do they work? [Online]. Available: <https://www.hackerone.com/blog/what-are-bug-bounties-and-how-do-they-work>
- [10] (2025) Huntr bug bounty platform. Accessed: 2025-09-20. [Online]. Available: <https://huntr.com/>
- [11] J. Bressers. (2025, May) Curl fights a flood of ai-generated bug reports from hackerone. Discusses the flood of AI-generated bug reports affecting open-source and bounty programs. [Online]. Available: <https://thenewstack.io/curl-fights-a-flood-of-ai-generated-bug-reports-from-hackerone/>
- [12] T. Claburn, “Open-source project curl is sick of users submitting ‘ai slop’ vulnerabilities,” *Ars Technica*, May 2025, reports on the surge of AI-generated ‘slop’ bug reports in the curl project. [Online]. Available: <https://arstechnica.com/gadgets/2025/05/open-source-project-curl-is-sick-of-users-submitting-ai-slop-vulnerabilities/>
- [13] D. Stenberg. (2025, Jul.) Death by a thousand slops. Blog post by curl’s founder describing the burden of AI-generated bug reports. [Online]. Available: <https://daniel.haxx.se/blog/2025/07/14/death-by-a-thousand-slops/>
- [14] K. Lawson. (2025, Oct.) Will ai-agent-fueled attacks force bug bounty programs to adapt? Analyzes AI-assisted bug hunting, new attack surfaces, and implications for bounty triage. [Online]. Available: <https://www.csoonline.com/article/4082265/ai-powered-bug-hunting-shakes-up-bounty-industry-for-better-or-worse.html>
- [15] M. Prins. (2025, Jul.) Beyond the noise: How hackerone cuts through the noise in the age of ai. HackerOne blog post on AI-assisted submissions and triage in the age of generative models. [Online]. Available: <https://www.hackerone.com/blog/beyond-the-noise-hai-triage-insight-agent>
- [16] HackerOne. (2025, October) Hackerone report finds 210% spike in ai vulnerability reports amid rise of ai autonomy. [Online]. Available: <https://www.hackerone.com/press-release/hackerone-report-finds-210-spike-ai-vulnerability-reports-amid-rise-ai-autonomy>
- [17] S. Hendrick and A. Ramaswami, “Maintainer perspectives on open source software security.”
- [18] M. Zhao, A. Laszka, T. Maillart, and J. Grossklags, “Crowdsourced security vulnerability discovery: Modeling and organizing bug-bounty programs,” in *The HCOMP Workshop on Mathematical Foundations of Human Computation*, Austin, TX, USA, 2016.
- [19] R. Ellis and Y. Stevens, “Bounty everything: Hackers and the making of the global bug marketplace,” 2022.
- [20] HackerOne. (2024) Report states. [Online]. Available: <https://docs.hackerone.com/en/articles/8475030-report-states>
- [21] ruisilva. Hackerone report #226514. [Online]. Available: <https://hackerone.com/reports/226514>
- [22] HackerOne. Palo alto software hackerone policy scopes. [Online]. Available: https://hackerone.com/palo_alto_software/policy_scopes
- [23] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” *arXiv preprint arXiv:1908.10084*, 2019.
- [24] xdemiray. Hackerone report #2215434. [Online]. Available: <https://hackerone.com/reports/2215434>
- [25] paramdham. Hackerone report #688546. [Online]. Available: <https://hackerone.com/reports/688546>
- [26] hackerboy404. Hackerone report #832593. [Online]. Available: <https://hackerone.com/reports/832593>
- [27] M. Hardt, E. Price, and N. Srebro, “Equality of opportunity in supervised learning,” *Advances in neural information processing systems*, vol. 29, 2016.
- [28] S. Mitchell, E. Potash, S. Barocas, A. D’Amour, and K. Lum, “Algorithmic fairness: Choices, assumptions, and definitions,” *Annual review of statistics and its application*, vol. 8, no. 1, pp. 141–163, 2021.
- [29] M. Zhao, A. Laszka, and J. Grossklags, “Devising effective policies for bug-bounty platforms and security vulnerability discovery,” *Journal of Information Policy*, vol. 7, pp. 372–418, 2017.
- [30] L. Zhang, E. M. Demirezen, and S. Kumar, “How to make my bug bounty cost-effective? a game-theoretical model,” *Information Systems Research*, vol. 36, no. 2, pp. 1031–1053, 2025.
- [31] T. Walshe and A. Simpson, “An empirical study of bug bounty programs,” in *2020 IEEE 2nd international workshop on intelligent bug fixing (IBF)*. IEEE, 2020, pp. 35–44.
- [32] L. Allodi et al., “A survey of bug bounty programs: Policies, effectiveness, and challenges,” *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–38, 2024.
- [33] O. Akgul, T. Eghtesad, A. Elazari, O. Gnawali, J. Grossklags, M. L. Mazurek, D. Votipka, and A. Laszka, “Bug {Hunters’} perspectives on the challenges and benefits of the bug bounty ecosystem,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 2275–2291.
- [34] G. Barre, “From breakers to builders: The role of bug bounty hunters in strengthening organizational cybersecurity,” *Information*, vol. 16, no. 3, p. 209, 2025.
- [35] J. Ayala, Y.-J. Tung, and J. Garcia, “A {Mixed-Methods} study of {Open-Source} software maintainers on vulnerability management and platform security features,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 2105–2124.
- [36] —, “Investigating vulnerability disclosures in open-source software using bug bounty reports and security advisories,” *arXiv preprint arXiv:2501.17748*, 2025.
- [37] J. Ayala, S. Ngo, and J. Garcia, “A deep dive into how open-source project maintainers review and resolve bug bounty reports,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 522–538.
- [38] M. Finifter, D. Akhawe, and D. Wagner, “An empirical study of vulnerability rewards programs,” in *22nd USENIX security symposium (USENIX Security 13)*, 2013, pp. 273–288.
- [39] F. Mohammad Ali Pour, M. Baradaran, and S. Ghasemshirazi, “The hacker’s guide to llms: Practical use cases and attack scenarios in bug bounty hunting,” *The Hacker’s Guide to LLMs: Practical Use Cases and Attack Scenarios in Bug Bounty Hunting*, 2024.